JAVA INTERFACES

Architecting Abstract Systems

codehive

WHAT IS AN INTERFACE?

An **Interface** in Java is a blueprint of a class. It is a powerful tool to achieve absolute abstraction. It provides a way to group related methods with empty bodies—defining **what** a class must do, but not **how** it should do it.

Think of it as a contract: any class that signs the contract (implements the interface) **must** provide the logic for the specific methods listed.

```
// Defining an Interface
interface Animal {
  public void animalSound(); // No body
  public void run();         // No body
}
```

IMPLEMENTING INTERFACES

To access interface methods, the interface must be "implemented" by another class using the implements keyword. The implementing class must provide a concrete implementation for **every** method defined in the interface.

```
class Pig implements Animal {
  public void animalSound() {
    System.out.println("The pig says: wee wee");
  }
  public void run() {
    System.out.println("The pig runs fast!");
  }
}

class Main {
  public static void main(String[] args) {
    Pig myPig = new Pig();
    myPig.animalSound();
  }
}
```

CRUCIAL NOTES & CONSTRAINTS

Interfaces have specific characteristics that distinguish them from normal classes and abstract classes:

Key Characteristics

No Objects: Interfaces cannot be instantiated directly. You cannot say `new Animal();`

Default Modifiers: All interface methods are implicitly `public` and `abstract`.

Constants: All attributes (variables) in an interface are implicitly `public`, `static`, and `final`.

No Constructors: An interface cannot contain a constructor because it cannot be instantiated.

MULTIPLE INTERFACES

While Java does **not** allow a class to inherit from more than one superclass (Multiple Inheritance), a single class **can** implement multiple interfaces. This is how Java handles complex behavior grouping.

```
interface FirstInterface {
  public void myMethod();
}

interface SecondInterface {
  public void myOtherMethod();
}

class DemoClass implements FirstInterface, SecondInterface {
  public void myMethod() { ... }
  public void myOtherMethod() { ... }
}
```

Separate multiple interfaces with a comma to extend the capabilities of your class.

SUMMARY & BEST PRACTICES

Use interfaces to define the behavior that several classes should have in common. This leads to cleaner, decoupled, and more testable code.

Abstraction Tool: Interfaces provide 100% abstraction (until Java 8 default methods).

Keyword: Use `interface` to define and `implements` to use.

Security: Protects internal implementation by exposing only method signatures.

Scaling: Makes it easy to swap implementations without changing the calling code.